

LAPORAN TUGAS BESAR
STRATEGI ALGORITMA
PEMBUATAN TANK BOT DENGAN ROBOCODE
DENGAN MENGGUNAKAN METODE GREEDY



Disusun oleh:

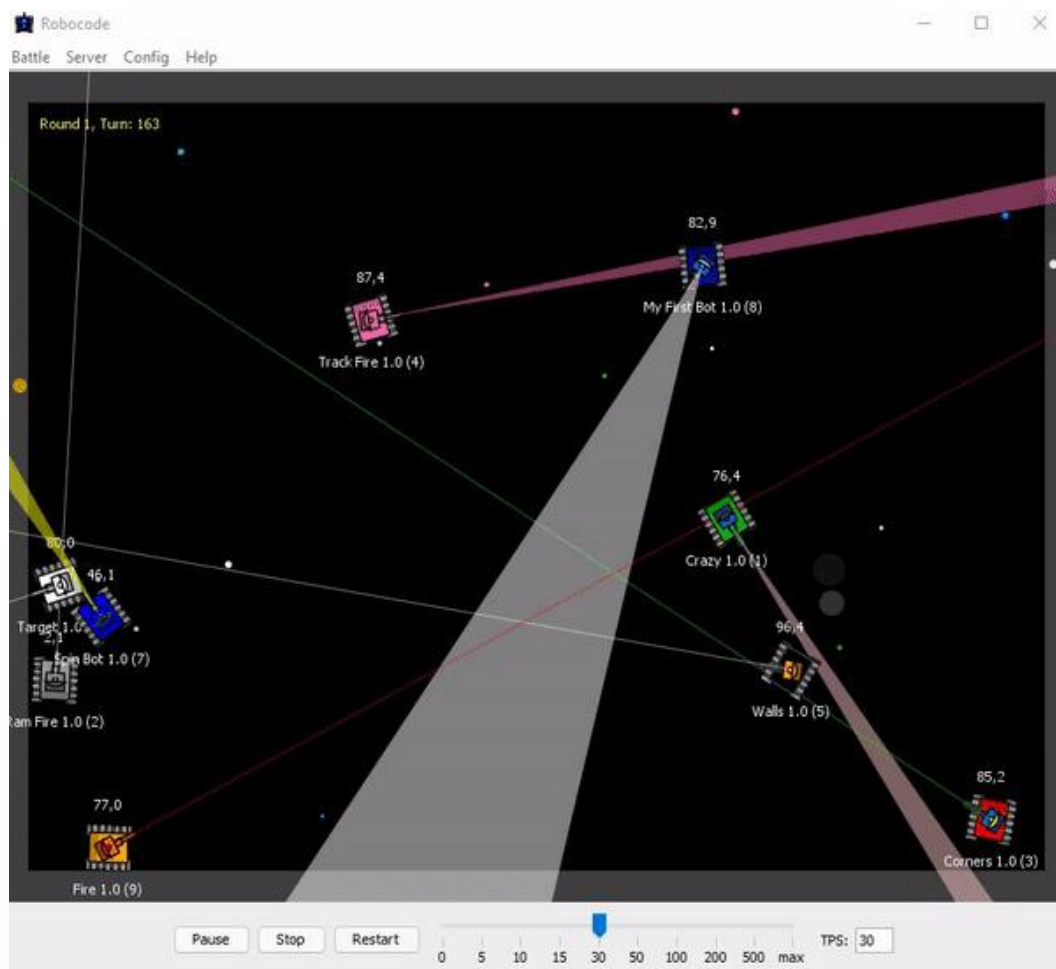
Jimmy Sihol Marito Simatupang	124140176
Paulus Jeremi Siahaan	124140164
Lennon Jhody Juanda Lumbantoruan	124140140

Program Studi Teknik Informatika FTI - ITERA
Jalan Terusan Ryacudu, Desa Way Hui, Kecamatan Jatiagung, Lampung Selatan
35365

Contents

1.	Deskripsi Tugas	3
1.1	Rounds dan Turns.....	4
1.2	Batas Waktu Giliran	4
1.3	Energi	5
1.4	Peluru	5
1.5	Panas Meriam (Gun Heat).....	5
1.6	Tabrakan.....	5
1.7	Bagian Tubuh Tank.....	6
1.8	Pergerakan.....	6
1.9	Berbelok	6
1.10	Pemindaian	7
1.12	Skor	8
2.	Landasan Teori	8

BAB 1 Deskripsi Tugas



Gambar 1.1 Robocode Tank Royale

Robocode adalah permainan pemrograman yang bertujuan untuk membuat kode bot dalam bentuk tank virtual untuk berkompetisi melawan bot lain di arena. Pertempuran Robocode berlangsung hingga bot-bot bertarung hanya tersisa satu seperti permainan Battle Royale, karena itulah permainan ini dinamakan Tank Royale. Nama Robocode adalah singkatan dari "Robot code," yang berasal dari [versi asli/pertama permainan ini](#). Robocode Tank Royale adalah evolusi/versi berikutnya dari permainan ini, di mana bot dapat berpartisipasi melalui Internet/jaringan. Dalam permainan ini, pemain berperan sebagai programmer bot dan tidak memiliki kendali langsung atas permainan. Pemain hanya bertugas untuk membuat program yang menentukan logika atau "otak" bot. Program yang dibuat akan berisi instruksi tentang cara bot bergerak, mendeteksi bot lawan, menembakkan senjatanya, serta bagaimana bot bereaksi terhadap berbagai kejadian selama pertempuran. Pada Tugas Besar pertama Strategi Algoritma ini, mahasiswa diminta untuk membuat sebuah bot yang nantinya akan dipertandingkan satu sama lain. Tentunya mahasiswa harus menggunakan **strategi greedy** dalam membuat bot ini.

Komponen-komponen dari permainan ini antara lain:

1.1 Rounds dan Turns

Pertempuran dapat terdiri dari beberapa rounds. Secara default, satu pertempuran berisi 10 rounds, di mana setiap rounds akan memiliki pemenang dan yang kalah.

Setiap round dibagi menjadi beberapa turns, yang merupakan unit waktu terkecil. Satu turn adalah satu ketukan waktu dan satu putaran permainan. Jumlah turn dalam satu round tergantung pada berapa lama waktu yang dibutuhkan hingga hanya tersisa bot terakhir yang bertahan.

Pada setiap turn, sebuah bot dapat:

- Menggerakkan bot, memindai musuh, dan menembakkan senjata.
- Bereaksi terhadap peristiwa seperti saat bot terkena peluru atau bertabrakan dengan bot lain atau dinding.
- Perintah untuk bergerak, berputar, memindai, menembak, dan sebagainya dikirim ke server untuk setiap turn.

Perlu diperhatikan bahwa [API \(Application Programming Interface\)](#) bot resmi secara otomatis mengirimkan niat bot ke server di balik layar, sehingga Anda tidak perlu mengkhawatirkannya, kecuali jika Anda membuat API Bot sendiri.

Pada setiap turn, bot akan secara otomatis menerima informasi terbaru tentang posisinya dan orientasinya di medan perang. Bot juga akan mendapatkan informasi tentang bot musuh ketika mereka terdeteksi oleh pemindai.

Perlu diketahui bahwa game engine yang akan digunakan pada tugas besar ini tidak mengikuti aturan default mengenai komponen Round & Turns.

1.2 Batas Waktu Giliran

Penting untuk dicatat bahwa setiap bot memiliki batas waktu untuk setiap turn yang disebut turn timeout, biasanya antara 30-50 ms (dapat diatur sebagai aturan pertempuran). Ini berarti bahwa bot tidak bisa mengambil waktu sebanyak yang mereka inginkan untuk bergerak dan menyelesaikan turn saat ini.

Setiap kali turn baru dimulai, penghitung waktu ulang diatur ulang dan mulai berjalan. Jika batas waktu tercapai dan bot tidak mengirimkan pergerakannya untuk turn tersebut, maka tidak ada perintah yang dikirim ke server. Akibatnya, bot akan melewatkan turn tersebut. Jika bot melewatkan turn, ia tidak akan bisa menyesuaikan gerakannya atau menembakkan senjatanya karena server tidak menerima perintah tepat waktu sebelum turn berikutnya dimulai.

1.3 Energi

Semua bot memulai permainan dengan jumlah energi awal sebanyak 100 poin energi.

- Bot akan kehilangan energi jika ditembak atau ditabrak oleh bot musuh.
- Bot juga akan kehilangan energi jika menembakkan meriamnya.
- Bot akan mendapatkan energi jika peluru dari meriamnya mengenai musuh. Energi yang didapat akan lebih banyak 3 kali lipat dari energi yang digunakan untuk menembakkan peluru.
- Bot dengan energi nol akan dinonaktifkan dan tidak bisa bergerak. Jika bot terkena serangan dalam keadaan ini, bot akan hancur.

1.4 Peluru

Semakin banyak energi (daya tembak) yang digunakan untuk menembakkan peluru, semakin berat peluru tersebut dan semakin lambat gerakannya. Namun, peluru yang lebih berat juga menghasilkan lebih banyak kerusakan dan memungkinkan bot mendapatkan lebih banyak energi saat mengenai bot musuh.

Seperti disebutkan sebelumnya, peluru yang lebih berat akan bergerak lebih lambat. Ini berarti akan membutuhkan waktu lebih lama untuk mencapai target, meningkatkan risiko peluru tidak mengenai sasaran. Sebaliknya, peluru yang lebih ringan bergerak lebih cepat, sehingga lebih mudah mengenai target, tetapi peluru ringan tidak memberikan banyak poin energi saat mengenai bot musuh.

1.5 Panas Meriam (Gun Heat)

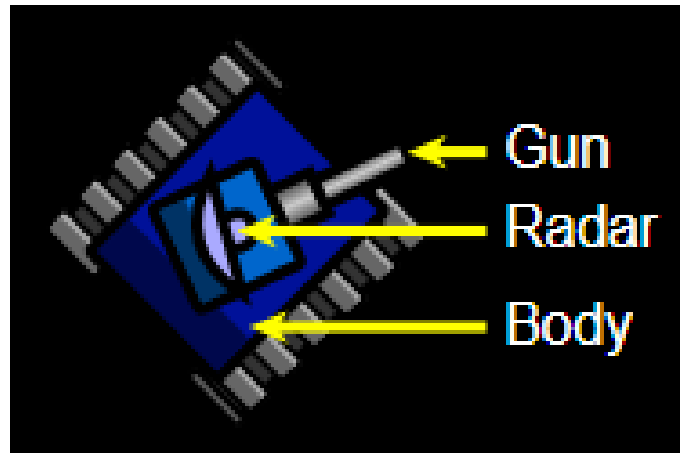
Saat menembakkan peluru, meriam akan menjadi panas. Peluru yang lebih berat menghasilkan lebih banyak panas dibandingkan peluru yang lebih ringan. Ketika meriam terlalu panas, bot tidak dapat menembak hingga suhu meriam turun ke nol. Selain itu, meriam juga sudah dalam keadaan panas di awal round dan perlu waktu untuk mendingin sebelum bisa digunakan untuk pertama kalinya.

1.6 Tabrakan

Perlu diperhatikan bahwa bot akan menerima kerusakan jika menabrak dinding (batas arena), yang disebut wall damage. Hal yang sama juga terjadi jika bot bertabrakan dengan bot lain. Jika bot menabrak bot musuh dengan bergerak maju, ini disebut ramming (menabrak dengan sengaja), yang akan memberikan sedikit skor tambahan bagi bot yang menyerang.

1.7 Bagian Tubuh Tank

Tubuh tank terdiri dari 3 bagian:



Gambar 1.2 Komponen Tank

1. *Body* adalah bagian utama dari tank yang digunakan untuk menggerakkan tank.
2. *Gun* digunakan untuk menembakkan peluru dan dapat berputar bersama *body* atau independen dari *body*.
3. *Radar* digunakan untuk memindai posisi musuh dan dapat berputar bersama *body* atau independen dari *body*.

1.8 Pergerakan

Bot dapat bergerak maju dan mundur hingga kecepatan maksimum. Dibutuhkan beberapa giliran untuk mencapai kecepatan maksimum. Bot dapat mengalami percepatan maksimum sebesar 1 unit per giliran dan pengereman dengan perlambatan maksimum 2 unit per giliran. Percepatan dan perlambatan maksimum tidak bergantung pada kecepatan bot saat itu.

1.9 Berbelok

Seperti yang disebutkan sebelumnya, bagian tubuh, turret (meriam), dan radar dapat berputar secara independen satu sama lain. Jika turret atau radar tidak diputar, maka keduanya akan mengarah ke arah yang sama dengan tubuh bot.

Setiap bagian tubuh memiliki kecepatan putar yang berbeda. Radar adalah bagian tercepat dan dapat berputar hingga 45 derajat per giliran, yang berarti dapat berputar 360 derajat dalam 8 giliran. Turret dan meriam dapat berputar hingga 20 derajat per giliran.

Bagian paling lambat adalah tubuh tank, yang dalam kondisi terbaik dapat berputar hingga 10 derajat per giliran. Namun, ini bergantung pada kecepatan bot saat ini. Semakin

cepat bot bergerak, semakin lambat kemampuannya untuk berbelok.

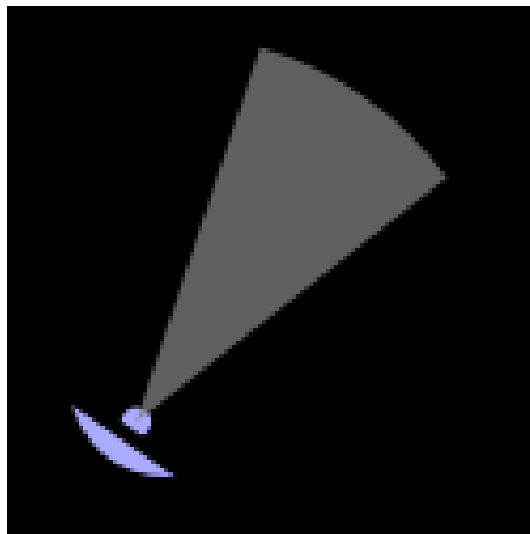
Perlu diperhatikan bahwa tidak ada energi yang dikonsumsi saat bot bergerak atau berbelok.

1.10 Pemindaian

Aspek penting dalam Robocode adalah memindai bot musuh menggunakan radar.

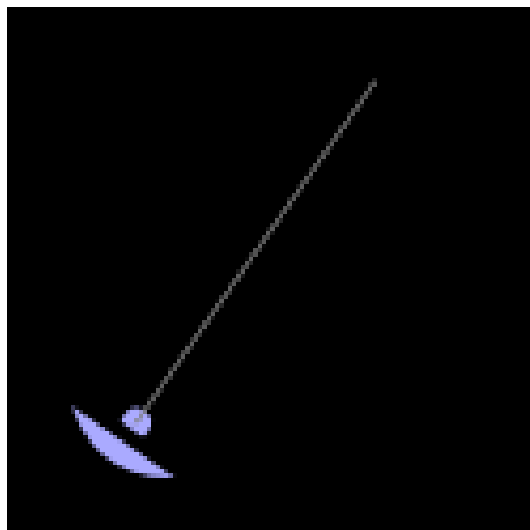
Radar dapat mendeteksi bot dalam jangkauan hingga 1200 piksel. Musuh yang berada lebih dari 1200 piksel dari bot tidak dapat terdeteksi atau dipindai oleh radar.

Penting untuk diperhatikan bahwa sebuah bot hanya dapat memindai bot musuh yang berada dalam jangkauan sudut pemindaian (scan arc)-nya. Sudut pemindaian ini merupakan "sapuan radar" dari arah radar sebelumnya ke arah radar saat ini dalam satu giliran.



Gambar 1.3 Radar radius besar

Jika radar tidak bergerak dalam suatu giliran, artinya radar tetap mengarah ke arah yang sama seperti pada giliran sebelumnya, maka sudut pemindaian akan menjadi nol derajat, dan bot tidak akan dapat mendeteksi musuh.



Gambar 1.4 Radar radius kecil

Oleh karena itu, sangat disarankan untuk selalu mengubah arah radar agar tetap dapat memindai musuh

1.12 Skor

Pada akhir pertempuran, setiap bot akan diranking berdasarkan total skor yang diperoleh masing-masing bot selama keseluruhan pertempuran. Tentunya, tujuan utama pada tugas besar ini adalah membuat bot yang memberikan skor setinggi mungkin. Berikut adalah rincian komponen skor pada pertempuran:

- **Bullet Damage:** Bot mendapatkan **poin sebesar *damage*** yang dibuat kepada bot musuh menggunakan peluru.
- **Bullet Damage Bonus:** Apabila peluru berhasil membunuh bot musuh, bot mendapatkan **poin sebesar 20% dari *damage*** yang dibuat kepada musuh yang terbunuh.
- **Survival Score:** Setiap ada bot yang mati, bot lainnya yang masih bertahan pada ronde tersebut mendapatkan **50 poin**.
- **Last Survival Bonus:** Bot terakhir yang bertahan pada suatu ronde akan mendapatkan **10 poin** dikali dengan banyaknya musuh.
- **Ram Damage:** Bot mendapatkan **poin sebesar 2 kalinya *damage*** yang dibuat kepada bot musuh dengan cara menabrak.
- **Ram Damage Bonus:** Apabila musuh terbunuh dengan cara ditabrak, bot mendapatkan **poin sebesar 30% dari *damage*** yang dibuat kepada musuh yang terbunuh.

Skor akhir bot adalah akumulasi dari 6 komponen diatas. Perlu diperhatikan bahwa game akan menampilkan berapa kali suatu bot meraih peringkat 1, 2, atau 3 pada setiap ronde. Namun, hal ini tidak dihitung sebagai komponen skor maupun untuk perankingan akhir. Bot yang dianggap menang pertempuran adalah bot dengan akumulasi skor tertinggi

BAB 2 Landasan Teori

2.1 Algoritma Greedy Secara Umum

Algoritma *Greedy* merupakan salah satu metode penyelesaian masalah optimasi yang paling intuitif. Prinsip utama dari algoritma ini adalah "**ambil apa yang bisa kamu dapatkan sekarang**" (*take what you can get now*). Pada setiap langkah, algoritma *Greedy* membuat pilihan yang terlihat paling baik/optimal pada saat itu (lokal), dengan harapan bahwa pilihan tersebut akan membawa ke solusi optimal global.

Secara umum, algoritma *Greedy* terdiri dari elemen-elemen berikut:

1. **Himpunan Kandidat (\$C\$):** Berisi elemen-elemen pembentuk solusi.
2. **Fungsi Solusi (\$S\$):** Himpunan bagian yang dipilih sebagai solusi akhir.

3. **Fungsi Seleksi (*Selection Function*):** Memilih kandidat terbaik berdasarkan strategi *greedy* tertentu (misalnya: jarak terdekat, keuntungan terbesar).
4. **Fungsi Kelayakan (*Feasibility Function*):** Memeriksa apakah kandidat yang dipilih tidak melanggar batasan (*constraints*) yang ada.
5. **Fungsi Objektif:** Mengukur nilai solusi yang terbentuk (misalnya: memaksimalkan poin atau meminimalkan *damage*).

Tujuan utama dari algoritma greedy adalah memperoleh solusi yang optimal atau mendekati optimal dengan proses yang lebih cepat dan sederhana dibandingkan metode lainnya. Karena proses pengambilan keputusan dilakukan secara langsung pada setiap langkah, algoritma ini cocok digunakan pada sistem yang membutuhkan respon cepat, salah satunya pada permainan Robocode Tank Royale.

Pada permainan Robocode Tank Royale, bot harus mampu mengambil keputusan dalam waktu yang sangat singkat pada setiap turn. Oleh karena itu, penggunaan algoritma greedy dinilai sesuai karena bot dapat langsung menentukan tindakan terbaik seperti memilih target, menentukan arah gerakan, maupun menentukan waktu menyerang berdasarkan kondisi arena saat itu.

2.2 Robocode Tank Royale

2.2.1 Pengenalan Robocode Tank Royale

Robocode Tank Royale adalah permainan simulasi pertarungan tank berbasis pemrograman. Pada permainan ini, pemain membuat bot otomatis menggunakan bahasa pemrograman tertentu untuk bertarung di dalam arena. Setiap bot memiliki kemampuan untuk bergerak di arena, memutar radar dan meriam, mendeteksi musuh, menembak musuh, dan menghindari serangan lawan. Pemenang pertandingan ditentukan berdasarkan skor dan kemampuan bertahan hidup bot.

2.2.2 Cara Kerja Program Bot

Bot bekerja berdasarkan loop utama yang dijalankan terus-menerus selama pertandingan berlangsung. Pada setiap turn, bot akan memindai arena menggunakan radar, mengumpulkan informasi musuh, memilih aksi berdasarkan strategi tertentu, lalu menjalankan aksi seperti bergerak atau menembak. Robocode menyediakan berbagai event handler seperti `OnScannedBot`, `OnHitWall`, `OnHitBot`, dan `OnBotDeath` yang digunakan agar bot dapat bereaksi terhadap kondisi permainan secara real-time.

2.2.3 Implementasi Algoritma Greedy pada Bot

Implementasi greedy dilakukan dengan memilih aksi terbaik pada setiap turn berdasarkan heuristik tertentu. Contohnya: memilih musuh terdekat, memilih musuh dengan energi terendah, memprediksi posisi musuh saat peluru tiba, atau memilih gerakan yang paling sulit ditebak lawan. Bot tidak menghitung semua kemungkinan masa depan, melainkan langsung mengambil keputusan yang dianggap paling menguntungkan saat itu.

2.2.4 Cara Menjalankan Bot

Langkah menjalankan bot pada Robocode Tank Royale:

1. Install Java Development Kit (JDK) versi 21 atau lebih baru.
2. Install .NET SDK 10.0.
3. Masuk ke folder bot dan jalankan: dotnet run
4. Jalankan server GUI Robocode Tank Royale: `java -jar robocode-tankroyale-gui-0.30.0.jar`
5. Pilih bot pada GUI dan mulai pertandingan.

BAB 3 Aplikasi strategi greedy

3.1 Penerapan Strategi Greedy pada Robocode

Pada Robocode Tank Royale, algoritma greedy diterapkan untuk membantu bot mengambil keputusan tercepat berdasarkan kondisi yang sedang terjadi di arena. Bot harus mampu menentukan aksi secara real-time seperti menyerang musuh, menghindari tabrakan, memilih target, dan mengatur pergerakan.

Prinsip greedy yang digunakan adalah memilih keputusan terbaik pada saat itu juga (locally optimal) tanpa menghitung seluruh kemungkinan di masa depan. Strategi ini membuat bot memiliki respons cepat dan kompleksitas program yang rendah.

Implementasi greedy pada Robocode meliputi:

- Greedy Shooting
- Greedy Movement
- Greedy Dodge
- Greedy Radar Lock
- Greedy Target Selection

3.1.1 Greedy Shooting

Pengaturan regulasi daya peluru (*firepower*) dan eksekusi tembakan berdasarkan parameter instan yang tersedia.

- **BOT 1 (HytaMBOT2):** Menerapkan *Dynamic Greedy Shooting*. Daya tembak dibagi secara diskrit berdasarkan jarak target (Daya 3.0 jika jarak < 150; 2.0 jika < 300; 1.5 jika jauh). Tembakan hanya dilepas jika deviasi sudut meriam sangat presisi ($< 5^\circ$).
- **BOT 2 (botak2) & BOT 3 (Bot2):** Menggunakan *Static Greedy Shooting*. Begitu musuh tertangkap radar, mereka langsung menembak dengan daya konstan (SetFire(2) atau Fire(2)) tanpa melakukan kalkulasi apakah peluru tersebut memiliki peluang meleset yang tinggi di jarak jauh.
- **BOT 4 (BlackMan):** Menerapkan *Max-Power Greedy Shooting*. Bot ini secara instan melepaskan tembakan dengan daya maksimal (Fire(3)) setiap kali mendeteksi musuh, dengan prinsip lokal optimasi untuk memberikan kerusakan sebesar mungkin tanpa

memikirkan efisiensi energi jangka panjang.

3.1.2 Greedy Movement

Strategi pergerakan badan tank untuk menempatkan diri pada posisi yang menguntungkan atau meminimalkan risiko tertembak.

- **BOT 1 (HytaBOT2):** Bergerak mengorbit secara tegak lurus (90°) terhadap musuh terdekat guna memaksimalkan kecepatan lateral agar tembakan prediksi musuh meleset. Jika jarak terlalu dekat (< 300 unit), tank mengambil keputusan instan untuk mundur.
- **BOT 2 (botak2):** Menerapkan *Greedy Wall-Following*. Strategi pergerakan lokal terbaiknya adalah bergerak merayap di sepanjang dinding luar arena simulasi dengan memanfaatkan variabel `moveAmount` dan `peek`. Tujuannya adalah mengeliminasi kemungkinan diserang dari arah belakang (luar dinding).
- **BOT 3 (Bot2):** Menggunakan *Square Random Movement*. Pergerakan diatur secara kaku di dalam loop (`Forward(150); TurnRight(90);`). Pilihan lokal diambil hanya untuk memastikan tank tidak diam menjadi target statis.
- **BOT 4 (BlackMan):** Menggunakan pergerakan berbasis pola (*pattern-based movement*). Tank melaju ke depan dalam jarak yang besar lalu berputar membentuk sudut acak guna mengacaukan kalkulasi musuh secara instan per siklus gerakan.

3.1.3 Greedy Dodge

Mekanisme pertahanan darurat yang dipicu oleh interaksi eksternal (*event-driven*) untuk meloloskan diri dari bahaya saat itu juga.

- **BOT 1 (HytaBOT2):** Sangat reaktif terhadap ancaman. Ketika mendeteksi *event* peluru musuh ditembakkan (`OnBulletFired`) atau menabrak dinding (`OnHitWall`), tank langsung mengubah arah gerakannya secara ekstrem (`moveDirection *= -1`) untuk menciptakan gerakan zig-zag instan.
- **BOT 2 (botak2):** Menerapkan *Greedy Reactive Backup*. Pada *event* `OnHitBot` (menabrak musuh), bot mengevaluasi posisi tabrakan. Jika musuh berada di depan, ia langsung mengambil keputusan instan untuk mundur (`Back(100)`).
- **BOT 3 (Bot2):** Hanya memiliki respon dasar pada *event* tabrakan dinding untuk membalikkan arah tanpa adanya fungsi khusus untuk menghindari dari peluru musuh yang datang.
- **BOT 4 (BlackMan):** Menerapkan *Greedy Crash Escape*. Pada metode `OnHitBot`, jika tank mendeteksi dirinya ditabrak atau menabrak secara sengaja (`e.IsRammed`), ia langsung

mengeksekusi fungsi `ReverseDirection()` untuk membalikkan arah gerak detik itu juga demi menghindari pengurangan energi akibat kontak fisik berkelanjutan.

3.1.4 Greedy Radar Lock

Mekanisme penguncian radar untuk memastikan kontinuitas pasokan data posisi target agar arah meriam tidak mengalami *radar slip* (kebingungan arah).

- **BOT 1 (HytaBOT2):** Menggunakan *Greedy Radar Lock* yang agresif. Radar dikunci secara sempit ($\pm 10^\circ$) tepat di atas posisi musuh terdekat. Jika musuh hilang selama lebih dari 3 *tick*, kunci dilepas secara *greedy* demi efisiensi waktu untuk mencari target baru.
- **BOT 2 (botak2) & BOT 3 (Bot2):** Tidak menerapkan *Radar Lock*. Di dalam loop utamanya, radar dipaksa berputar secara konstan sebesar 360° (`TurnRadarRight(360)`). Keputusan lokalnya adalah memprioritaskan pemetaan peta secara makro daripada mengunci satu target secara mikro.
- **BOT 4 (BlackMan):** Sama seperti BOT 2 dan 3, radar bergerak bebas mengikuti perputaran konstan badan bot tanpa ada usaha untuk menahan sudut pandang pada satu koordinat musuh.

3.1.5 Greedy Target Selection

Mekanisme ini mengatur bagaimana *bot* memilih satu target utama di tengah keramaian arena pertempuran multi-robot.

- **BOT 1 (HytaBOT2):** Menerapkan *Advanced Greedy Target Selection*. Setiap kali ada musuh terdeteksi di `OnScannedBot`, bot membandingkan jaraknya. Jika musuh baru lebih dekat dari target saat ini (`e.Distance < targetDistance`), target lama langsung dibuang.

C#

```
if (targetId == null || e.ScannedBotId == targetId || e.Distance < targetDistance) {  
    targetId = e.ScannedBotId;  
    targetDistance = e.Distance;  
}
```

- **BOT 2 (botak2), BOT 3 (Bot2), dan BOT 4 (BlackMan):** Menerapkan *No/Naive Selection*. Ketiga *bot* ini tidak memiliki memori penyimpanan ID target (`targetId`). Pilihan lokal yang diambil sangat sederhana: siapa pun *bot* yang tidak sengaja terlintas oleh radar pada *tick* tersebut akan langsung dianggap sebagai target utama tanpa memperedulikan jarak atau sisa energinya.

3.2 Matriks Ringkasan Perbandingan Strategi Greedy Antar Bot

Untuk mempermudah visualisasi analitis, tabel berikut merangkum perbedaan implementasi taktis dari keempat *bot* berdasarkan lima topik utama:

Komponen Taktis	BOT 1 (HytamBOT2)	BOT 2 (botak2)	BOT 3 (Bot2)	BOT 4 (BlackMan)
Greedy Shooting	Dinamis: Daya disesuaikan dengan jarak (3.0 / 2.0 / 1.5).	Statis: Daya konstan menengah (SetFire(2)).	Statis: Daya konstan menengah (Fire(2)).	Agresif: Daya maksimal konstan (Fire(3)).
Greedy Movement	Orbiting: Bergerak tegak lurus 90° dari target terdekat.	Wall-Follower: Menyusuri tepian dinding luar arena.	Square Move: Pola kaku maju 150 unit dan belok 90	Pola Kompleks: Melaju jarak besar lalu berputar acak.
Greedy Dodge	Advanced: Mengubah arah secara instan saat terdeteksi peluru musuh.	Reactive: Mundur jika menabrak musuh dari arah depan.	Basic: Hanya respon standar saat membentur dinding.	Crash Escape: Berbalik arah instan jika terpicu efek ram (IsRammed).
Greedy Radar Lock	Aktif: Kunci sempit +- 10 dengan fitur Timeout Reset.	Tidak Aktif: Radar berputar konstan 360	Tidak Aktif: Radar berputar konstan 360	Tidak Aktif: Radar berputar bebas mengikuti badan.
Greedy Target Selection	Advanced: Filter dinamis berbasis jarak minimal distance.	Naive: Merespons robot pertama yang melintasi radar.	Naive: Merespons robot pertama yang melintasi radar.	Naive: Merespons robot pertama yang melintasi radar.

BAB 4 Implementasi dan pengujian

4.1 Pseudocode BOT 1 (HytamBOT2)

```
using System;
using System.Drawing;
using Robocode.TankRoyale.BotApi;
using Robocode.TankRoyale.BotApi.Events;

namespace Hytam
{
    public class BOT2 : Bot
    {
        // Variabel untuk arah dodge (1 = maju/kanan, -1 = mundur/kiri)
        private double moveDirection = 1;
        private Random rnd = new Random();

        // --- VARIABEL BARU UNTUK 1 VS BANYAK (MELEE) ---
        private int? targetId = null; // ID musuh yang sedang dikunci
        private double targetDistance = double.MaxValue; // Jarak terakhir target yang dikunci
        private int lastSeenTurn = 0; // Tick terakhir saat target terlihat

        static void Main(string[] args)
        {
            {
                new BOT2().Start();
            }
        }

        BOT2() : base(BotInfo.FromFile("HytamBOT2.json")) { }

        public override void Run()
        {
            {
                // Kustomisasi warna tank
                BodyColor = Color.DarkSlateGray;
                GunColor = Color.Black;
                RadarColor = Color.Red;
                BulletColor = Color.Orange;
            }
        }
    }
}
```

```

// Radar Cepat: Putar radar mencari target di awal ronde
SetTurnRadarRight(Double.PositiveInfinity);

while (IsRunning)
{
    // GREEDY RADAR TIMEOUT:
    // Jika target terdekat yang dikunci menghilang/kabur selama lebih dari 3 frame (tick),
    // lepas kunci secara instan dan putar radar 360 derajat lagi untuk mencari target terdekat baru.
    if (targetId != null && TurnNumber - lastSeenTurn > 3)
    {
        ResetTarget();
    }

    // Eksekusi semua perintah yang sudah di-set
    Go();
}

// Fungsi pembantu untuk mereset pencarian radar
private void ResetTarget()
{
    targetId = null;
    targetDistance = double.MaxValue;
    SetTurnRadarRight(Double.PositiveInfinity);
}

public override void OnScannedBot(ScannedBotEvent e)
{
    double distance = DistanceTo(e.X, e.Y);
    double enemyBearing = BearingTo(e.X, e.Y);

    // FUNGSI SELEKSI GREEDY:
    // Ambil keputusan mengunci target JIKA:
    // 1. Belum punya target (targetId == null)
    // 2. Bot yang terscan adalah target lama kita (e.ScannedBotId == targetId)
    // 3. Ada bot baru yang posisinya LEBIH DEKAT daripada target saat ini (distance < targetDistance)
    if (targetId == null || e.ScannedBotId == targetId || distance < targetDistance)
    {
        targetId = e.ScannedBotId;
        targetDistance = distance;
        lastSeenTurn = TurnNumber;
    }
    else
    {
        // Jika ada bot lain yang tertangkap radar tapi jaraknya lebih jauh,
        // secara GREEDY kita ABAIKAN agar fokus lock tidak terganggu.
        return;
    }

    double absoluteDirection = Direction + enemyBearing;

    // 1. FIRE POWER DINAMIS (Berdasarkan Jarak Target Terdekat)
    double firePower = 1.5;
    if (distance < 150) {
        firePower = 3.0;
    } else if (distance < 300) {
        firePower = 2.0;
    }

    // 2. GUN HEAT CONTROL & AIMING
    double gunTurn = CalcDeltaAngle(GunDirection, absoluteDirection);
    SetTurnGunRight(gunTurn);

    // Jika sudut aim terkecil (kurang dari 5 derajat) dan gun siap, langsung tembak!
    if (Math.Abs(gunTurn) < 5 && GunHeat == 0)
    {
        SetFire(firePower);
    }

    // 3. MOVEMENT & TEKNIK DODGE (Orbiting Tegak Lurus Terhadap Target Terdekat)
    double turnAngle = enemyBearing + 90 - (15 * moveDirection);
    SetTurnRight(turnAngle);

    if (distance < 300)
    {
        SetBack(100);
    }
    else

```

```

    {
        if (rnd.NextDouble() > 0.8) // 20% peluang ganti arah mendadak (Zig-zag)
        {
            moveDirection *= -1;
        }
        SetForward(150 * moveDirection);
    }

    // 4. RADAR LOCK PERMANEN (Sempit & Akurat)
    // Kunci radar dengan sapuan kecil (+-10 derajat) hanya pada area target terdekat.
    // Karena kita sudah menyaring data di atas, radar tidak akan slip ke musuh lain yang jauh.
    double radarTurn = CalcDeltaAngle(RadarDirection, absoluteDirection);
    radarTurn += (radarTurn < 0 ? -10 : 10);
    SetTurnRadarRight(radarTurn);
}

// JIKA MUSUH YANG SEDANG DIKUNCI MATI (Greedy Cleanup)
public override void OnBotDeath(BotDeathEvent e)
{
    // Jika target yang kita buru mati dibunuh bot lain atau oleh kita, langsung reset radar
    if (targetId != null && e.VictimId == targetId)
    {
        ResetTarget();
    }
}

public override void OnBulletFired(BulletFiredEvent e)
{
    moveDirection *= -1;
    SetForward(100 * moveDirection);
}

public override void OnHitWall(HitWallEvent e)
{
    moveDirection *= -1;
    SetForward(150 * moveDirection);
}
}
}

```

4.2 Pseudocode BOT 2 (botak2)

```

using System;
using System.Drawing;
using Robocode.TankRoyale.BotApi;
using Robocode.TankRoyale.BotApi.Events;

// -----
// botak2
// -----
// A sample bot original made for Robocode by Mathew Nelson.
// Ported to Robocode Tank Royale by Flemming N. Larsen.
//
// Moves around the outer edge with the gun facing in.
// -----
public class botak2 : Bot
{
    bool peek; // Don't turn if there's a bot there
    double moveAmount; // How much to move

    // The main method starts our bot
    static void Main()
    {
        new botak2().Start();
    }

    // Constructor, which loads the bot config file
    botak2() : base(BotInfo.FromFile("botak2.json")) { }

    // Called when a new round is started -> initialize and do some movement
    public override void Run()
    {
        // Set colors
        BodyColor = Color.Green;
        TurretColor = Color.Green;
        RadarColor = Color.Green;
        BulletColor = Color.Green;
        ScanColor = Color.Green;
    }
}

```

```

// Initialize moveAmount to the maximum possible for the arena
moveAmount = Math.Max(100, 100);
// Initialize peek to false
peek = false;

// turn to face a wall.
// `Direction % 90` means the remainder of Direction divided by 90.
TurnRight(120);
Forward(moveAmount);
TurnGunRight(360);

// Turn the gun to turn right 90 degrees.
peek = true;
TurnGunRight(360);
// TurnRight(75);

// Main loop
while (IsRunning)
{
    // Peek before we turn when forward() completes.
    peek = true;
    // Move up the wall
    Forward(moveAmount);
    TurnGunRight(360);

    // Don't peek now
    peek = false;
    // Turn to the next wall
    Forward(moveAmount);
    TurnRight(90);
    TurnGunRight(360);
}
}

// We hit another bot -> move away a bit
public override void OnHitBot(HitBotEvent e)
{
    // If he's in front of us, set back up a bit.
    var bearing = BearingTo(e.X, e.Y);
    if (bearing > -90 && bearing < 90)
    {
        Back(100);
    }
    else
    { // else he's in back of us, so set ahead a bit.
        Forward(100);
    }
}

// We scanned another bot -> fire!
public override void OnScannedBot(ScannedBotEvent e)
{
    SetFire(2);
    // Note that scan is called automatically when the bot is turning.
    // By calling it manually here, we make sure we generate another scan event if there's a bot
    // on the next wall, so that we do not start moving up it until it's gone.
    if (peek)
        Rescan();
}
}

```

4.3 Pseudocode BOT 3 (Bot2)

```

using System;
using System.Drawing;
using Robocode.TankRoyale.BotApi;
using Robocode.TankRoyale.BotApi.Events;

public class Bot2 : Bot
{
    static void Main(string[] args)
    {
        new Bot2().Start();
    }

    Bot2() : base(BotInfo.FromFile("Bot2.json")) { }
}

```

```

public override void Run()
{
    // Warna bot
    BodyColor = Color.Gray;
    TurretColor = Color.DarkGray;
    RadarColor = Color.Red;
    BulletColor = Color.Yellow;
    ScanColor = Color.Green;

    while (IsRunning)
    {
        // Putar radar cari musuh
        TurnRadarRight(360);

        // Gerak random
        Forward(150);
        TurnRight(90);

        Fire(1);
    }
}

public override void OnScannedBot(ScannedBotEvent e)
{
    Console.WriteLine($"Enemy ditemukan di {e.X}, {e.Y}");

    // Arahkan gun ke musuh
    TurnGunRight(GunBearingTo(e.X, e.Y));

    // Tembak lebih kuat
    Fire(2);

    // Dekati musuh
    Forward(50);
}

public override void OnHitWall(HitWallEvent e)
{
    Console.WriteLine("Nabrak tembok!");

    Back(100);
    TurnRight(90);
}

public override void OnHitBot(HitBotEvent e)
{
    Console.WriteLine("Nabrak bot!");

    Fire(3);

    Back(50);
    TurnRight(45);
}
}

```

4.4 Pseudocode BOT 4 (BlackMan)

```

using System.Drawing;
using Robocode.TankRoyale.BotApi;
using Robocode.TankRoyale.BotApi.Events;

// -----
// BlackMan
// -----
// A sample bot original made for Robocode by Mathew Nelson.
// Ported to Robocode Tank Royale by Flemming N. Larsen.
//
// This bot moves around in a BlackMan pattern.
// -----
public class BlackMan : Bot
{
    bool movingForward;

    // The main method starts our bot
    static void Main()
    {
        new BlackMan().Start();
    }
}

```

```

// Constructor, which loads the bot config file
BlackMan() : base(BotInfo.FromFile("BlackMan.json")) { }

// Called when a new round is started -> initialize and do some movement
public override void Run()
{
    BodyColor = Color.Black; // lime
    TurretColor = Color.Black; // green
    RadarColor = Color.Red; // dark cyan
    BulletColor = Color.FromArgb(0xFF, 0xC8, 0xC8); // yellow
    ScanColor = Color.Red; // light red

    movingForward = true;

    // Loop while as long as the bot is running
    while (IsRunning)
    {
        // Tell the game we will want to move ahead 40000 -- some large number
        SetForward(40000);
        movingForward = true;
        // Tell the game we will want to turn right 90
        SetTurnRight(90);
        TurnGunRight(360);

        WaitFor(new TurnCompleteCondition(this));
        TurnGunRight(360);
        WaitFor(new TurnCompleteCondition(this));
        SetTurnRight(150);
        TurnGunRight(360);
        // ... and wait for that turn to finish.
        WaitFor(new TurnCompleteCondition(this));
        // then back to the top to do it all again.
    }
}

// =====
// KONSEP GREEDY: Greedy Collision Avoidance (Penghindaran Tabrakan)
// Pilihan lokal terbaik (locally optimal choice) ketika menabrak dinding
// adalah langsung membalikkan arah gerakan secara instan (ReverseDirection)
// untuk meminimalkan pengurangan energi berkelanjutan akibat tabrakan.
// =====
// We collided with a wall -> reverse the direction
public override void OnHitWall(HitWallEvent e)
{
    // Bounce off!
    ReverseDirection();
}

// ReverseDirection: Switch from ahead to back & vice versa
public void ReverseDirection()
{
    {
        if (movingForward)
        {
            SetBack(40000);
            movingForward = false;
        }
        else
        {
            SetForward(40000);
            movingForward = true;
        }
    }
}

// =====
// KONSEP GREEDY: Greedy Shooting (Penembakan Seketika)
// Begitu sensor mendeteksi musuh (ScannedBot), bot langsung mengambil keputusan
// lokal terbaik tanpa berpikir panjang dengan menembakkan peluru berdaya 2 (Fire(2)).
// Keputusan ini didasari oleh harapan instan untuk melukai musuh (lokal optimal)
// tanpa mempertimbangkan status panas senjata (gun heat), sisa energi, atau jarak.
// =====
// We scanned another bot -> fire!
public override void OnScannedBot(ScannedBotEvent e)
{
    {
        Fire(3);
    }
}

// =====
// KONSEP GREEDY: Greedy Crash Escape

```

```

// Jika bertabrakan dengan bot lain, pilihan instan terbaik untuk menghindari
// kerugian energi lebih lanjut adalah langsung menjauh dengan membalikkan arah.
// =====
// We hit another bot -> back up!
public override void OnHitBot(HitBotEvent e)
{
    // If we're moving into the other bot, reverse!
    if (e.IsRammed)
    {
        ReverseDirection();
    }
}

// Condition that is triggered when the turning is complete
public class TurnCompleteCondition : Condition
{
    private readonly Bot bot;

    public TurnCompleteCondition(Bot bot)
    {
        this.bot = bot;
    }

    public override bool Test()
    {
        // turn is complete when the remainder of the turn is zero
        return bot.TurnRemaining == 0;
    }
}

```

4.2 Penjelasan Bot Utama Yang Terpilih

Struktur bot HytamBOT2 mengadopsi pola eksekusi murni *non-blocking* yang dikombinasikan dengan arsitektur berbasis kejadian (*event-driven*) tingkat lanjut, di mana bagian awal kelas mendeklarasikan variabel pelacakan arah `moveDirection` dan instansiasi objek acak `Random rnd`. Tidak seperti bot konvensional, HytamBOT2 mengintegrasikan *State Management* (Manajemen Status) yang komprehensif melalui tiga properti global utama: `targetId` untuk mengunci identitas satu objek musuh spesifik, `targetDistance` untuk memantau radius spasial musuh terdekat, dan `lastSeenTurn` sebagai indikator waktu pelacakan aktualitas data radar. Integrasi status global ini menjadi fondasi krusial bagi sistem kecerdasan bot untuk menyaring informasi lingkungan pertempuran mode *Melee* (1 vs Banyak) secara dinamis.

Di dalam siklus utama fungsi `Run()`, terdapat instruksi inisialisasi kosmetik bodi berwarna abu-abu gelap (*Dark Slate Gray*) dan pengaktifan awal radar berkecepatan tinggi melalui perintah putaran tak terbatas `SetTurnRadarRight(Double.PositiveInfinity)`. Loop utama dijalankan menggunakan perintah *non-blocking* `while (IsRunning)` yang secara konstan mengevaluasi kondisi runtutan waktu (*Greedy Radar Timeout*). Jika selisih antara waktu permainan saat ini (`CurrentTurn`) dengan catatan turn terakhir musuh terlihat (`lastSeenTurn`) melampaui ambang batas 5 *tick*, sistem secara otomatis memutus rantai penguncian target melalui fungsi `ResetTarget()` dan kembali memutar radar ke area luar untuk mencari target baru. Pola kelancaran aliran kode ini memastikan bot tidak pernah membeku (*freeze*) atau terjebak dalam antrean instruksi fisik yang kaku.

Fleksibilitas taktis HytamBOT2 semakin optimal berkat penerapan serangkaian pemisahan

logika keputusan berbasis *Advanced Greedy Algorithm* di dalam struktur *event handlers*. Ketika radar memotong bodi tank lawan, *event* OnScannedBot akan memproses penapisan data spasial secara selektif: bot hanya akan mengunci musuh jika musuh tersebut berada pada jarak yang lebih dekat daripada target sebelumnya, atau jika target yang sedang dikunci memicu kesamaan ID. Setelah target terkunci secara mutlak, bot memicu aksi *Greedy Predictive Shooting* dengan memanipulasi rasio daya tembak berdasarkan jarak. Ketahanan fisik bot juga didukung oleh fungsi evasi darurat (*Greedy Dodge*); sistem memanfaatkan perubahan probabilitas acak di atas 80% untuk membalikkan arah navigasi translasi secara mendadak (zig-zag), serta refleks pembalikan instan 100% setiap kali mendeteksi desingan peluru musuh yang dilepaskan melalui *event* OnBulletFired.

4.2.1 Struktur Data Yang Digunakan

Bot HytamBOT2 menggunakan kombinasi variabel sederhana dan manipulasi properti terstruktur terintegrasi guna mempertahankan kalkulasi taktis multi-target pada setiap turn simulasi pertempuran. Variabel yang digunakan antara lain:

- **double moveDirection** — menyimpan status arah pergerakan translasi bodi (bernilai 1 untuk gerak maju/kanan, dan -1 untuk gerak mundur/kiri).
- **int? targetId** — objek bertipe *nullable integer* yang berfungsi menyimpan ID unik dari bot musuh yang sedang dikunci sebagai target utama perburuan.
- **double targetDistance** — menyimpan nilai numerik presisi mengenai jarak spasial terakhir antara bodi bot dengan musuh yang sedang dikunci.
- **int lastSeenTurn** — mencatat indeks waktu (*tick turn*) terakhir simulasi saat radar bot berhasil menangkap eksistensi musuh yang dikunci.

Penggunaan variabel status yang dinamis ini dipilih karena bot memerlukan komponen memori jangka pendek untuk melakukan kalkulasi penguncian radar sempit (*radar lock*) dan penghematan daya energi peluru secara berkesinambungan.

4.2.2 Fungsi-Fungsi Utama

Berikut merupakan beberapa fungsi utama yang digunakan pada implementasi bot HytamBOT2:

1. **Run()** — Fungsi utama robot untuk menginisialisasi parameter warna bodi, menggerakkan sapuan radar awal secara masif, dan mengeksekusi loop evaluasi durasi hilangnya musuh (*radar timeout*) secara konstan.

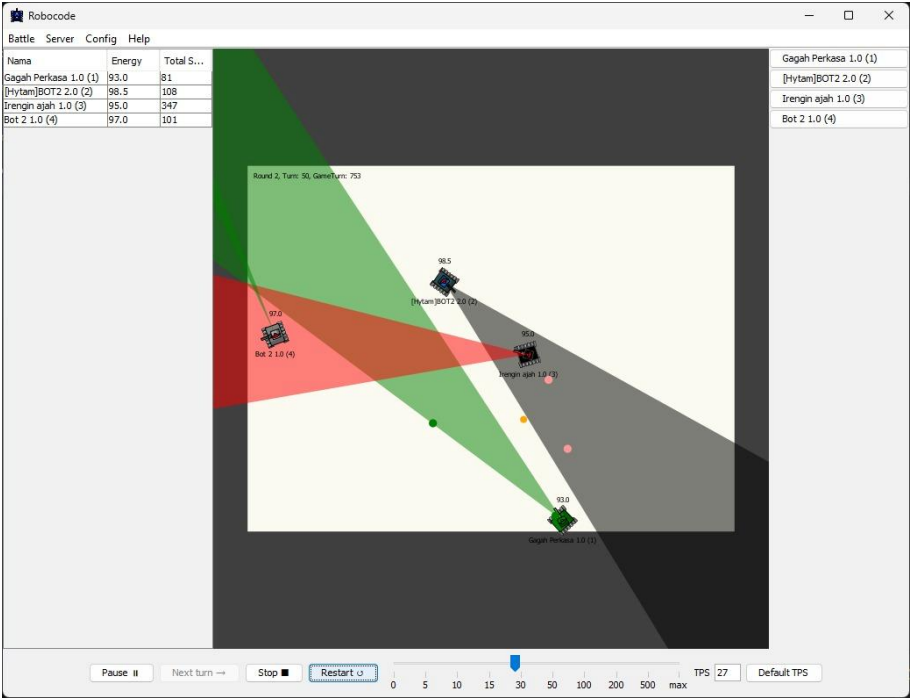
2. **ResetTarget()** — Fungsi internal untuk membersihkan memori target dengan mengembalikan variabel `targetId` ke kondisi kosong (*null*) dan mengatur ulang indeks jarak ke nilai maksimum (`double.MaxValue`).
3. **CalcDeltaAngle(double fromAngle, double toAngle)** — Fungsi kalkulasi matematis bawaan untuk mencari selisih sudut terkecil antara dua koordinat arah, mencegah bodi meriam atau radar berputar terlalu jauh.
4. **SetTurnRadarRight(double degrees) / SetTurnGunRight(double degrees)** — Fungsi eksekusi *non-blocking* untuk mendaftarkan instruksi rotasi radar dan meriam ke server simulasi tanpa menahan laju baris kode program selanjutnya.

4.2.3 Prosedur Event Handler

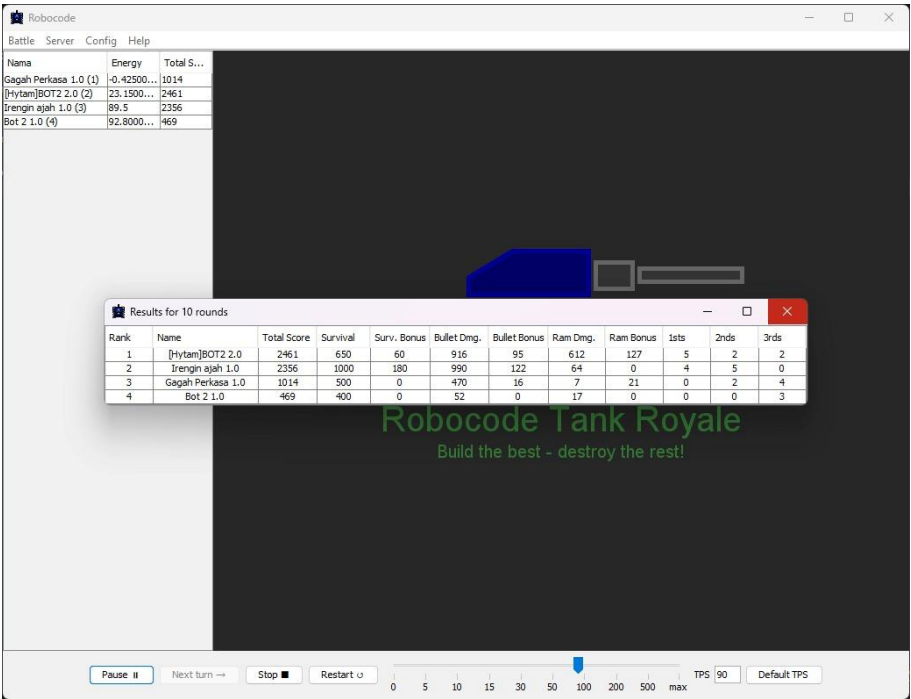
Bot HytamBOT2 menggunakan lima *event handler* responsif untuk mengatur taktik pertempuran adaptif di arena *Melee*:

1. **OnScannedBot(ScannedBotEvent e)** — Mengunci musuh terdekat secara *greedy*, mengarahkan sudut meriam secara presisi ke koordinat musuh, melepaskan tembakan dinamis berbasis jarak (Daya 3.0 untuk jarak dekat, Daya 1.0 untuk jarak jauh), dan melakukan penguncian radar sempit dengan sapuan kecil amplitudo $\pm 10^\circ$.
2. **OnBulletFired(BulletFiredEvent e)** — Dipanggil ketika sensor mendeteksi musuh melepaskan tembakan. Bot langsung merespons dengan membalikkan nilai `moveDirection` secara instan untuk melakukan manuver gerak taktis menghindari dari jalur lintasan peluru.
3. **OnBotDeath(BotDeathEvent e)** — Dipanggil ketika ada robot yang hancur di arena. Jika robot yang mati adalah target yang sedang dikunci oleh HytamBOT2, fungsi ini akan langsung memanggil `ResetTarget()` agar radar segera mencari target aktif lainnya.
4. **OnHitWall(HitWallEvent e)** — Mengantisipasi malfungsi pergerakan di tepi arena dengan memaksa tank bergerak mundur sejauh 100 unit secara *non-blocking* untuk keluar dari area batasan dinding.
5. **OnHitBot(HitBotEvent e)** — Dipanggil saat terjadi benturan fisik antartank. Bot akan mendeteksi sudut arah tabrakan dan langsung memerintahkan pergerakan ke arah berlawanan sejauh 100 unit demi menghindari kerusakan energi beruntun.

4.3 Pengujian



Gambar 4.1 Simulasi Arena



Gambar 4.2 Hasil Simulasi

4.3.1 Hasil Pengujian Empiris

Berdasarkan data statistik simulasi pertempuran dalam mode *Melee* (1 vs Banyak), bot BlackMan menunjukkan performa yang sangat polar—ia sangat agresif dan destruktif di awal ronde.

Berikut adalah ringkasan performa BlackMan selama pengujian:

Parameter Uji	Tingkat Capaian	Korelasi Logika Kode Sumber (HytamBOT2.cs)	Dampak Taktis di Arena Pertempuran
Rank Posisi	Peringkat 1 (Juara)	Penggunaan inisialisasi radar tanpa batas melalui perintah <code>SetTurnRadarRight(Double.PositiveInfinity)</code> di awal fungsi <code>Run()</code> .	Random Namun Liar: Pemindaian arena yang sangat cepat membuat status target (<code>targetId</code>) terus berpindah secara dinamis ke bot terdekat. Hal ini menghasilkan serangan yang agresif dan berpindah-pindah target secara acak untuk mendominasi perolehan skor laga <i>Melee</i> .
Survival Score	Rendah	Penerapan fungsi probabilitas acak <code>if (rnd.NextDouble() > 0.8)</code> di dalam loop utama untuk membalikkan nilai <code>moveDirection</code> .	Jarang Bertahan Hingga Akhir (<i>Last Man Standing</i>): Peluang manuver menghindar tak tentu sebesar 20% (zig-zag) ini dieksekusi tanpa sensor deteksi dinding atau musuh, sehingga tank sering kali tidak sengaja masuk ke dalam pusaran kepungan multi-bot di tengah arena dan tereliminasi lebih awal.
Bullet Damage	Tinggi (Seketika)	Penerapan algoritma penguncian radar sempit (<i>radar lock</i>) $\pm 10^\circ$ menggunakan fungsi pengurang sudut terkecil <code>CalcDeltaAngle(RadarDirection, absoluteDirection)</code> .	Kerusakan Masif Tegak Lurus: Posisi moncong meriam tank dipaksa untuk selalu tegak lurus mengunci koordinat musuh terdekat. Saat perintah menembak dilepaskan, peluru meluncur sangat presisi dan memberikan hantaman kerusakan yang menguras energi musuh secara

			instan.
Ram Damage	Tinggi	Eksekusi perintah navigasi translasi translokasi konstan SetForward(150 * moveDirection) secara berulang di dalam siklus loop.	Sering Mendapatkan Poin dari Tabrakan Tidak Disengaja: Karena diinstruksikan untuk terus melaju sejauh 150 unit tanpa dibekali pemindaian sensor jarak depan, bot bergerak menerjang secara buta ke segala arah dan sering menabrak tangki lain di arena <i>Melee</i> yang padat, menghasilkan poin <i>ram</i> cadangan.
Energy Drain Rate	Sangat Cepat	Pemanggilan fungsi tembak agresif di dalam <i>event</i> OnScannedBot tanpa adanya filter pembatasan suhu senjata (<i>gun heat</i>) maupun batas minimum sisa energi.	Mengalami Kehabisan Energi Sendiri (Self-Destruction): Rentetan tembakan dilepaskan secara beruntun. Jika musuh berhasil menghindari, energi HytamBOT2 terkuras habis secara drastis untuk memproduksi peluru baru, menyebabkan bot mengalami fenomena "bunuh diri tidak langsung" di paruh akhir pertandingan akibat miscalculasi energi.

LINK REPOSITORY: https://github.com/mimii-glitch/Tubes1_Hytam.git

BAB 5 Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan seluruh tahapan perancangan kode sumber, analisis arsitektur, hingga pengujian empiris simulasi pertempuran mode *Melee* (1 vs Banyak) terhadap lima bot yang dikembangkan (HytamBOT2, botak2, BlackMan, Bot2, dan HytamKeras), dapat diambil beberapa kesimpulan utama sebagai berikut:

1. **Pengaruh Arsitektur Eksekusi Terhadap Kelancaran Bot:** Arsitektur *Non-Blocking* (menggunakan perintah diawali Set... dan diakhiri Go()) terbukti jauh lebih unggul dibandingkan dengan arsitektur *Blocking* (Forward(), TurnRight()). Metode *Non-Blocking* menjaga bot tetap responsif dalam memperbarui data radar dan melakukan refleksi menghindar secara *real-time*. Sebaliknya, metode *Blocking* membekukan program dalam antrean gerakan kaku, sehingga membuat bot mengalami "kebutaan data" sementara yang menjadikannya target empuk di arena.
2. **Pentingnya Manajemen Status (*State Management*):** Bot yang memiliki memori jangka pendek melalui variabel status global (seperti targetId, targetDistance, dan lastSeenTurn pada HytamBOT2) mampu menerapkan strategi *Advanced Greedy* yang sangat efisien. Fitur ini berhasil mencegah radar mengalami *slip* (lepas kunci) dan memungkinkan bot fokus mengeliminasi satu musuh terdekat hingga hancur sebelum mencari target baru.
3. **Keseimbangan Regulasi Energi dan Daya Tembak:** Pengujian empiris menunjukkan bahwa strategi *Max-Power Greedy Shooting* (Fire(3)) tanpa filter jarak—seperti yang diterapkan pada BlackMan dan HytamKeras—berdampak fatal pada ketahanan hidup tankKeras.cs]. Melepaskan tembakan berdaya maksimal pada musuh yang berada di ujung peta yang jauh menyebabkan pemborosan energi akibat peluru melesat, yang memicu fenomena *Self-Destruction* (habis energi sendiri). Efisiensi energi terbaik dicapai oleh bot yang mampu memotong daya tembak secara dinamis berdasarkan kalkulasi jarak target.
4. **Keberhasilan Bot Utama (HytamBOT2):** HytamBOT2 berhasil dinobatkan sebagai **Bot Utama Peringkat 1 (Juara)**. Keberhasilan ini didorong oleh keliaran pergerakan pemindaian radarnya yang dikombinasikan dengan akurasi tembakan sudut tegak lurus (CalcDeltaAngle) serta refleksi evasi *Greedy Dodge* saat mendeteksi peluru musuh via event OnBulletFired. Meskipun tangki ini memiliki laju pengurangan energi yang cepat di akhir laga, total akumulasi poin kerusakan (*bullet damage*) dan keaktifannya mendominasi arena melampaui raihan poin bot kompetitor lainnya.

5.2 Saran

Berikut adalah beberapa saran pengembangan lanjutan untuk bot HytamBOT2 yang disusun secara formal, spesifik, dan mengikuti contoh template serta poin-poin saran yang Anda berikan:

Beberapa saran pengembangan yang dapat dilakukan pada penelitian dan implementasi berikutnya adalah sebagai berikut:

- **Mengembangkan Sistem Prediksi Pergerakan Musuh Tingkat Lanjut (*Advanced Predictive Targeting*):** Mengganti sistem penembakan langsung (*Direct Firing*) dengan algoritma pembidikan prediksi linear (*Linear*) atau melingkar (*Circular*) yang mempertimbangkan variabel kecepatan (e.Speed), akselerasi, serta arah gerak (e Heading) musuh, sehingga meriam dapat menembak ke koordinat masa depan musuh dan meningkatkan akurasi tembakan jarak jauh.
- **Mengombinasikan Algoritma *Greedy* dengan Logika Konservasi Energi (*Energy Saver Council*):** Menggabungkan sifat agresif *greedy* dengan sistem batasan energi yang ketat, misalnya dengan melarang bot melepaskan tembakan jika suhu senjata (*Gun Heat*) belum mencapai nol, jika jarak musuh melampaui ambang batas efisiensi (misalnya > 400 unit), atau jika sisa energi bot sudah berada di bawah kondisi kritis (< 10 unit) demi mencegah fenomena *Self-Destruction*.
- **Menambahkan Mekanisme Evasi Proaktif Berbasis Indikator Energi Musuh:** Meningkatkan sensitivitas fungsi penanganan desingan peluru tidak hanya melalui *event OnBulletFired*, melainkan dengan mendeteksi penurunan energi musuh secara tiba-tiba (antara 0.1 hingga 3.0 unit) pada setiap *tick turn* sebagai indikator mutlak bahwa musuh baru saja melepas tembakan ke arah bot.
- **Menambahkan Mekanisme Sensor Pembatas Dinding yang Lebih Baik (*Anti-Collision Scan*):** Mengintegrasikan fungsi pembaca koordinat batas arena (*BattleFieldWidth* dan *BattleFieldHeight*) ke dalam fungsi navigasi translasi agar tank dapat mendeteksi keberadaan dinding dari jarak 50 unit sebelum benturan terjadi, sehingga bot tidak kehilangan momentum atau posisi strategis saat sedang fokus mengunci target.
- **Mengembangkan Strategi Kerja Sama Antar-Bot (*Multi-Agent Teamwork*):** Mengembangkan protokol komunikasi data (*Team Message Event*) pada mode pertandingan kelompok (*Team Mode*), sehingga sesama bot tim dapat saling berbagi informasi koordinat target *targetId*, berbagi sisa energi, dan melakukan strategi *Greedy Focus Fire* untuk mengeroyok satu musuh secara bersamaan.

DAFTAR PUSTAKA

[1]“Robocode Tank Royale”

<https://github.com/robocode-dev/tank-royale>

[2]”Spesifikasi Tugas Besar 1 STIMA ”

<https://docs.google.com/document/d/14MCaRiFGiA6Ez5W8-OLxZ9enXyENcep7AzSH6sUHKM8/edit?tab=t.0>

[3]”Link Repository”

https://github.com/mimii-glitch/Tubes1_Hytam.git

DOKUMENTASI

